

Várias classes, um só main.py

Importando, misturando objetos de tipos diferentes, e um primeiro laço de jogo.

RESUMO, SE VOCÊ SÓ TEM 1 MINUTO

Um projeto de verdade quase nunca tem uma classe só. Esse guia mostra como importar várias classes num `main.py`, colocar objetos de tipos diferentes na mesma lista, e chamar métodos sem se importar muito de qual classe exata é cada objeto — desde que todos tenham um método com o mesmo nome. No final tem um esqueleto de batalha por turnos pra continuar no joguinho.

O QUE TEM AQUI

1. Por que separar em arquivos (recap rápido)
2. Importando mais de uma classe
3. Uma lista com objetos de classes diferentes
4. Chamando métodos sem se importar da classe exata
5. Uma novidade: combinando condições com `and`
6. Um laço de jogo simples
7. Esqueleto: batalha por turnos
8. Erros comuns
9. Perguntas frequentes
10. Checklist mental
11. Glossário rápido

1. Por que separar em arquivos (recap rápido)

Como já vimos, cada classe geralmente ganha o seu próprio arquivo, e um `main.py` importa e organiza tudo:

```
projeto/
├─ personagem.py
├─ guerreiro.py
├─ mago.py
└─ main.py
```

Até agora, os exemplos usaram só uma classe importada por vez. Esse guia é sobre o que muda quando o `main.py` usa *várias* classes ao mesmo tempo.

2. Importando mais de uma classe

Cada classe vem de um `import` próprio, geralmente um por linha:

```
from personagem import Personagem
from guerreiro import Guerreiro
from mago import Mago
```

Não tem limite de quantas classes um `main.py` pode importar. O nome antes do `import` é sempre o nome do arquivo (sem `.py`); o nome depois é o nome da classe dentro daquele arquivo.

3. Uma lista com objetos de classes diferentes

Python não liga que os objetos numa lista venham de classes diferentes — uma lista aceita qualquer coisa, misturada:

```
thoric = Guerreiro("Thoric", 100)
elara = Mago("Elara", 60)

time = [thoric, elara]
```

`time` é uma lista comum, igual às que já apareceram com `Inimigo` no Dia 2 — só que agora com dois tipos de objeto diferentes dentro dela ao mesmo tempo.

4. Chamando métodos sem se importar da classe exata

```
for personagem in time:
    personagem.mostrar_status()
```

Esse `for` funciona pros dois objetos, mesmo `thoric` sendo um `Guerreiro` e `elara` sendo um `Mago` — desde que **as duas classes tenham um método chamado** `mostrar_status`. O laço não sabe (e não precisa saber) qual classe exata é cada objeto; ele só chama o método pelo nome, e cada objeto responde do seu próprio jeito.

*Isso tem nome — **polimorfismo** — mas guarda esse termo pra quando a gente falar dos três pilares. Por enquanto, o que importa é sentir como funciona na prática: métodos com o mesmo nome, em classes diferentes, podem ser chamados do mesmo jeito.*

Se uma das classes *não* tiver esse método, o laço quebra na hora de chegar nela — mais sobre isso na seção de erros comuns.

5. Uma novidade: combinando condições com `and`

Um laço de jogo geralmente precisa checar mais de uma condição ao mesmo tempo — por exemplo, "enquanto os dois personagens estiverem vivos". Pra isso existe o `and`: combina duas condições, e só é verdadeiro se **as duas** forem verdadeiras.

```
thoric.vida > 0 and inimigo.vida > 0
```

Se qualquer um dos dois lados for falso, o resultado inteiro é falso. Existe também o `or`, que é verdadeiro se **pelo menos um** dos lados for verdadeiro — não vamos precisar dele agora, mas é bom já saber que existe.

6. Um laço de jogo simples

Juntando `while`, `and`, e métodos de mais de um objeto:

```
turno = 1
while thoric.vida > 0 and inimigo.vida > 0:
    print("Turno", turno)
    inimigo.receber_dano(thoric.ataque)

    if inimigo.vida > 0:
        thoric.receber_dano(inimigo.ataque)

    turno += 1

if thoric.vida > 0:
    print(thoric.nome, "venceu!")
else:
    print(inimigo.nome, "venceu!")
```

Reparem: o `if inimigo.vida > 0` dentro do laço existe pra evitar que o inimigo já derrotado ainda consiga contra-atacar no mesmo turno em que morreu. É um detalhe pequeno, mas sem ele o resultado do "duelo" fica estranho.

7. Esqueleto: batalha por turnos

ESQUELETO PRA COMPLETAR, NÃO O JOGO PRONTO

Juntando tudo — vários arquivos, várias classes, uma lista, e o laço de jogo — isso é o ponto de partida pro joguinho do Dia 3:

```
from guerreiro import Guerreiro
from mago import Mago
from inimigo import Inimigo

thoric = Guerreiro("Thoric", 100)
inimigo = Inimigo("Goblin", "pequeno")
inimigo.vida = 40
inimigo.ataque = 8

turno = 1
while thoric.vida > 0 and inimigo.vida > 0:
    print("--- Turno", turno, "---")
    inimigo.vida -= thoric.ataque
    print(inimigo.nome, "levou", thoric.ataque, "de dano.")

    if inimigo.vida > 0:
        thoric.vida -= inimigo.ataque
        print(thoric.nome, "levou", inimigo.ataque, "de dano.")

    turno += 1

if thoric.vida > 0:
    print(thoric.nome, "venceu a batalha!")
else:
    print(inimigo.nome, "venceu a batalha!")
```

Esse esqueleto ainda é simples de propósito — dano fixo, sem itens, sem escolha do jogador a cada turno. A ideia é vocês pegarem essa base e irem complicando aos poucos: talvez deixar o jogador escolher atacar ou fugir a cada turno, talvez variar o dano, talvez incluir mais de um inimigo na lista. É o mesmo espírito dos exercícios do Dia 2 — uma base pronta, e vocês constroem em cima.

8. Erros comuns

Esquecer de importar uma classe usada no main.py

CÓDIGO COM ERRO

```
from guerreiro import Guerreiro

magol = Mago("Elara", 60)
```

O QUE O PYTHON MOSTRA

```
NameError: name 'Mago' is not defined
```

CORREÇÃO

```
from guerreiro import Guerreiro
from mago import Mago

magol = Mago("Elara", 60)
```

Chamar um método que uma das classes não tem

CÓDIGO COM ERRO

```
for personagem in [guerreiro1, inimigo1]:
    personagem.conjurar_feitico()
```

O QUE O PYTHON MOSTRA

```
AttributeError: 'Inimigo' object has no attribute
'conjurar_feitico'
```

CORREÇÃO

```
for personagem in [guerreiro1, inimigo1]:
    personagem.mostrar_status() # metodo que os dois tem
```

O laço da seção 4 só funciona se **todo mundo na lista** tiver o método chamado. Antes de escrever um `for` assim, vale conferir se todas as classes envolvidas realmente têm aquele método com esse nome exato.

Laço infinito por esquecer de mudar o valor da condição

CÓDIGO COM ERRO

```
while thoric.vida > 0 and inimigo.vida > 0:
    print("Turno", turno)
    turno += 1
    # esqueceu de aplicar dano em algum lugar
```

O QUE ACONTECE

```
(o laço nunca termina — vida nunca muda)
```

CORREÇÃO

```
while thoric.vida > 0 and inimigo.vida > 0:
    inimigo.vida -= thoric.ataque
    turno += 1
```

Todo `while` precisa que algo dentro do laço mude o valor que a condição está checando — a mesma regra do laço de "vida decrescente" do Dia 1, só que agora a condição depende de dois objetos ao mesmo tempo.

9. Perguntas frequentes

Posso colocar dois objetos da mesma classe na mesma lista?

Sim, sem problema nenhum — `[inimigo1, inimigo2, inimigo3]` é perfeitamente válido, como já vimos no Dia 2.

E se as classes tiverem métodos com nomes diferentes, tipo `atacar_com_espada()` e `conjurar_feitico()`?

Aí não dá pra chamar todo mundo genericamente no mesmo `for` — cada classe precisa ser tratada separadamente (com `if isinstance(...)`, que a gente ainda não viu, ou com listas separadas por tipo). Padronizar os nomes dos métodos entre classes parecidas é justamente um dos motivos pra usar herança mais pra frente.

O `main.py` pode ter mais de uma lista?

Sim — por exemplo, uma lista `time` e uma lista `inimigos`, separadas, cada uma com seus próprios objetos.

10. Checklist mental

Antes de rodar um `main.py` com várias classes, confira:

- Toda classe usada tem um `from arquivo import Classe` correspondente?
- Se um `for` chama o mesmo método em vários objetos, todas as classes envolvidas realmente têm esse método?
- Todo `while` tem, dentro dele, algo que muda o valor checado na condição?
- Condições combinadas usam `and/or` corretamente, sem trocar um pelo outro?

11. Glossário rápido

and
combina duas condições; só é verdadeiro se as duas forem verdadeiras.

or
combina duas condições; é verdadeiro se pelo menos uma das duas for verdadeira.

AttributeError
erro que aparece ao tentar usar um atributo ou método que aquele objeto não tem.

polimorfismo
(prévia) quando objetos de classes diferentes respondem ao mesmo nome de método, cada um do seu jeito.

O esqueleto da seção 7 é o ponto de partida oficial do joguinho do Dia 3 — não precisa (nem deve) sair reescrevendo do zero.